FACHOBERSCHULE BERUFSOBERSCHULE BAMBERG	Informatik		Datum:
	Name:	Klasse:	

Lektion 2: Operatoren und Datentypen – Musterlösung

Do Now

Richtig ist A. Die Variable wird durch ihren Wert ersetzt, `print()` erzeugt eine Leerzeile, und `print(name)` gibt den Wert ohne Anführungszeichen aus.

- B prüft die Vorstellung, der Variablenname selbst werde ausgegeben.
- C prüft, ob `print()` ohne Inhalt als wirkungslos gilt.
- D prüft, ob das Leerzeichen im Text `"Hallo "` übersehen wurde.

Predict (Vorhersagen)

Erwartet wird fast immer 8. Tatsächlich erscheint `Die Summe ist: 53`. Das Programm läuft fehlerfrei und liefert trotzdem ein falsches Ergebnis – genau das ist der Anlass der Stunde.

Run (Ausführen)

Keine Fehlermeldung, aber die beiden Eingaben werden aneinandergehängt statt addiert.

Investigate (Untersuchen)

1. `print(type(zahl1))` liefert `<class 'str'>`. Die Eingabe ist Text, obwohl eine Ziffer eingetippt wurde: `input()` liefert immer einen String.
2. Weil beide Operanden Text sind, verkettet das + sie, statt zu addieren. Aus `"5"` und `"3"` wird `"53"`. Der Operator richtet sich nach dem Datentyp der Operanden.
3. Ergebnisse:

Ausdruck	Ergebnis	Bedeutung des Operators
`7 / 2`	`3.5`	Division, liefert immer eine Gleitkommazahl
`7 // 2`	3	Ganzzahldivision, der Nachkommaanteil entfällt
`7 % 2`	1	Rest der Ganzzahldivision (Modulo)
`7 ** 2`	49	Potenz, hier 7 hoch 2

4. `type(7 / 2)` liefert `float`, `type(7 // 2)` liefert `int`. Auffällig ist, dass / auch bei glatt aufgehender Division eine Gleitkommazahl zurückgibt: `4 / 2` ergibt `2.0`.

Modify (Verändern)

1. Umwandlung mit `int()` – der TypeError erscheint

```
zahl1 = int(input("Erste Zahl: "))
zahl2 = int(input("Zweite Zahl: "))
summe = zahl1 + zahl2
print("Die Summe ist: " + summe)   # TypeError in dieser Zeile
```

Die Meldung lautet sinngemäß: `can only concatenate str (not "int") to str`. Jetzt wird zwar richtig gerechnet, aber die Ausgabe scheitert, weil Text und Zahl nicht mit + verbunden werden können.

2. Reparatur mit `str()`

```
print("Die Summe ist: " + str(summe))
```

3. Dasselbe als f-String

```
print(f"Die Summe ist: {summe}")
```

Der f-String setzt den Wert aus den geschweiften Klammern ein und wandelt ihn selbst in Text um. Beide Schreibweisen sind ab jetzt gleichwertig.

4. Differenz und Produkt

```
differenz = zahl1 - zahl2
produkt = zahl1 * zahl2
print(f"Die Differenz ist: {differenz}")
print(f"Das Produkt ist: {produkt}")
```

Make (Erstellen)

```
laenge = int(input("Rohrlänge in cm: "))
stueck = int(input("Stücklänge in cm: "))

anzahl = laenge // stueck
rest = laenge % stueck

print(f"Es entstehen {anzahl} Stücke, es bleiben {rest} cm Rest.")
```

Akzeptiert wird jede Lösung, die `//` und `%` verwendet. Wer mit `/` und anschließendem `int()` arbeitet, kommt bei der Stückzahl ebenfalls zum Ziel, braucht für den Rest aber einen Umweg – das ist ein guter Gesprächsanlass.

Erweiterung

```
verschnitt = round(rest / laenge * 100, 1)
print(f"Verschnitt: {verschnitt} %")
```

Bei 250 cm und 40 cm sind das `10 / 250 * 100` = 4.0 Prozent. Typischer Fehler: `rest // laenge` statt `/` – dann kommt immer 0 heraus, weil die Ganzzahldivision den Nachkommaanteil verwirft.

Hinweise zur Besprechung

- Für die Schlussbesprechung eignet sich eine Lösung, die `//` und `%` sauber trennt – daran lassen sich die vier Begriffe der Stunde in zwei Minuten benennen.
- Ergiebig ist auch eine Lösung ohne `int()` bei der Eingabe: Sie scheitert bereits an der Rechnung und führt direkt zum Kernsatz der Stunde zurück.
- Mögliche Exit-Frage: Welchen Datentyp liefert `7 / 2`? Sie prüft den Punkt, der erfahrungsgemäß am schwächsten sitzt.